

第十二课预习报告

diceroll.c

- 作用: 该文件用于实现掷骰子的核心逻辑, 包括随机数生成与计数。
- 功能: 通过 `roll_n_dice(int dice, int sides)` 函数, 模拟 `dice` 个 `sides` 面的骰子点数总和。
- 演示效果:

```
1  /* diceroll.c -- dice role simulation */
2  /* compile with mandydice.c          */
3  #include "diceroll.h"
4  #include <stdio.h>
5  #include <stdlib.h>          /* for library rand() */
6
7  int roll_count = 0;          /* external linkage */
8
9  static int rollem(int sides) /* private to this file */
10 {
11     int roll;
12
13     roll = rand() % sides + 1;
14     ++roll_count;             /* count function calls */
15
16     return roll;
17 }
18
19 int roll_n_dice(int dice, int sides)
20 {
21     int d;
22     int total = 0;
23     if (sides < 2)
24     {
25         printf("Need at least 2 sides.\n");
26         return -2;
27     }
28     if (dice < 1)
29     {
30         printf("Need at least 1 die.\n");
```

```

31         return -1;
32     }
33
34     for (d = 0; d < dice; d++)
35         total += rollem(sides);
36
37     return total;
38 }

```

```

1  $ gcc manydice.c diceroll.c -o manydice && echo "6`n3`n0" | ./manydice
2  Enter the number of sides per die, 0 to stop.
3  How many dice?
4  You have rolled a 9 using 3 6-sided dice.
5  How many sides? Enter 0 to stop.
6  The rollem() function was called 3 times.
7  GOOD FORTUNE TO YOU!

```

where.c

- 作用: 分析程序中不同类型变量的内存位置, 展示静态区、堆区、栈区、常量区的存储分布。
- 功能: 打印各种变量（静态、自动、动态分配、常量字符串）的值与地址, 帮助理解内存布局在运行时的差异。
- 演示效果:

```

1  // where.c -- where's the memory?
2
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <string.h>
6
7  int static_store = 30;
8  const char * pcg = "String Literal";
9  int main()
10 {
11     int auto_store = 40;
12     char auto_string[] = "Auto char Array";
13     int * pi;
14     char * pcl;
15
16     pi = (int *) malloc(sizeof(int));

```

```

17     *pi = 35;
18     pcl = (char *) malloc(strlen("Dynamic String") + 1);
19     strcpy(pcl, "Dynamic String");
20
21     printf("static_store: %d at %p\n", static_store, &static_store);
22     printf("  auto_store: %d at %p\n", auto_store, &auto_store);
23     printf("          *pi: %d at %p\n", *pi, pi);
24     printf("  %s at %p\n", pcg, pcg);
25     printf(" %s at %p\n", auto_string, auto_string);
26     printf("  %s at %p\n", pcl, pcl);
27     printf("  %s at %p\n", "Quoted String", "Quoted String");
28     free(pi);
29     free(pcl);
30
31     return 0;
32 }

```

```

1  $ gcc where.c -o where && ./where
2  static_store: 30 at 00007ff621279000
3      auto_store: 40 at 000000d5c4bffa9c
4          *pi: 35 at 000002c026fe9c60
5      String Literal at 00007ff62127a000
6      Auto char Array at 000000d5c4bffa9d
7      Dynamic String at 000002c026fe9c80
8      Quoted String at 00007ff62127a06e

```

rand0.c

- 作用: 提供一个基于 ANSI C 标准的伪随机数生成算法（线性同余法）。
- 功能: 该文件的主要函数 `rand0()` 用于生成随机整数，算法依赖静态种子 `next`，实现跨平台的可移植随机数生成。
- 主要错误:
编译阶段出现 `undefined reference to WinMain`，原因是文件中没有 `main()` 函数；此文件为函数库文件，应与驱动程序（如 `r_drive0.c`）联合编译。
- 演示效果:

```

1  /* rand0.c -- produces random numbers          */
2  /*          uses ANSI C portable algorithm    */
3  static unsigned long int next = 1; /* the seed */
4
5  unsigned int rand0(void)
6  {
7      /* magic formula to generate pseudorandom number */
8      next = next * 1103515245 + 12345;
9      return (unsigned int) (next/65536) % 32768;
10 }

```

```

1  $ gcc rand0.c -o rand0 && ./rand0
2  undefined reference to WinMain
3  # 正确方式是:
4  $ gcc r_drive0.c rand0.c -o r_drive0 && ./r_drive0
5  16838
6  5758
7  10113
8  17515
9  31051

```

s_and_r.c

- 作用: 实现自定义随机数生成函数 `rand1()` 以及用于设置种子的函数 `srand1()`。
- 功能: 通过线性同余算法 (Linear Congruential Generator, LCG) 生成可重复的伪随机数序列。文件不包含 `main()` 函数, 仅作为函数库, 与 `r_drive1.c` 一起编译和运行。
- 主要错误:
单独编译时报错 `undefined reference to WinMain`, 因该文件为库函数文件, 没有入口函数。
- 演示效果:

```

1  /* s_and_r.c -- file for rand1() and srand1()   */
2  /*          uses ANSI C portable algorithm    */
3  static unsigned long int next = 1; /* the seed */
4
5  int rand1(void)
6  {
7      /* magic formula to generate pseudorandom number */

```

```

8     next = next * 1103515245 + 12345;
9     return (unsigned int) (next/65536) % 32768;
10 }
11
12 void srand1(unsigned int seed)
13 {
14     next = seed;
15 }

```

```

1 $ gcc s_and_r.c -o s_and_r && ./s_and_r
2 undefined reference to WinMain
3 # 正确方式应为:
4 $ gcc r_drive1.c s_and_r.c -o r_drive1 && echo "4`n7`nq" | ./r_drive1
5 Please enter your choice for seed.
6 1817
7 24166
8 10491
9 3711
10 15407
11 Please enter next seed (q to quit):
12 19564
13 9806
14 10868
15 22674
16 32531
17 Please enter next seed (q to quit):
18 Done

```

r_drive0.c

- 作用: 用于测试 `rand0()` 伪随机数生成函数的输出。
- 功能: 调用 `rand0()` 5 次并打印其返回的随机整数, 用以验证随机数生成算法的效果。
- 演示效果:

```

1 /* r_drive0.c -- test the rand0() function */
2 /* compile with rand0.c */
3 #include <stdio.h>
4 extern unsigned int rand0(void);
5
6 int main(void)

```

```

7  {
8      int count;
9
10     for (count = 0; count < 5; count++)
11         printf("%d\n", rand0());
12
13     return 0;
14 }

```

```

1  $ gcc r_drive0.c rand0.c -o r_drive0 && ./r_drive0
2  16838
3  5758
4  10113
5  17515
6  31051

```

r_drive1.c

- 作用: 测试改进版随机数生成器 `rand1()` 与种子设置函数 `srand1()` 的行为。
- 功能: 用户输入种子值, 程序利用种子初始化并生成一系列伪随机数, 演示种子如何影响随机数序列。
- 演示效果:

```

1  /* r_drive1.c -- test rand1() and srand1() */
2  /* compile with s_and_r.c */
3  #include <stdio.h>
4  #include <stdlib.h>
5  extern void srand1(unsigned int x);
6  extern int rand1(void);
7
8  int main(void)
9  {
10     int count;
11     unsigned seed;
12
13     printf("Please enter your choice for seed.\n");
14     while (scanf("%u", &seed) == 1)
15     {
16         srand1(seed);    /* reset seed */
17         for (count = 0; count < 5; count++)

```

```

18         printf("%d\n", rand1());
19         printf("Please enter next seed (q to quit):\n");
20     }
21     printf("Done\n");
22
23     return 0;
24 }

```

```

1  $ gcc r_drive1.c s_and_r.c -o r_drive1 && echo "4`n7`nq" | ./r_drive1
2  Please enter your choice for seed.
3  1817
4  24166
5  10491
6  3711
7  15407
8  Please enter next seed (q to quit):
9  19564
10 9806
11 10868
12 22674
13 32531
14 Please enter next seed (q to quit):
15 Done

```

parta.c

- 作用: 演示 C 语言中不同存储类别变量（自动变量、寄存器变量和外部变量）的使用与作用范围。
- 功能: 与 `partb.c` 联合编译，通过输入整数触发循环以展示各类变量的存储特性。外部变量 `count` 用于记录循环执行次数。
- 演示效果:

```

1  // parta.c --- various storage classes
2  // compile with partb.c
3  #include <stdio.h>
4  void report_count();
5  void accumulate(int k);
6  int count = 0;           // file scope, external linkage
7
8  int main(void)
9  {

```

```

10     int value;          // automatic variable
11     register int i;    // register variable
12
13     printf("Enter a positive integer (0 to quit): ");
14     while (scanf("%d", &value) == 1 && value > 0)
15     {
16         ++count;        // use file scope variable
17         for (i = value; i ≥ 0; i--)
18             accumulate(i);
19         printf("Enter a positive integer (0 to quit): ");
20     }
21     report_count();
22
23     return 0;
24 }
25
26 void report_count()
27 {
28     printf("Loop executed %d times\n", count);
29 }

```

```

1  $ gcc parta.c partb.c -o partab && echo "3`n5`n0" | ./partab
2  Enter a positive integer (0 to quit): loop cycle: 1
3  subtotal: 6; total: 6
4  Enter a positive integer (0 to quit): loop cycle: 2
5  subtotal: 15; total: 21
6  Enter a positive integer (0 to quit): Loop executed 2 times

```

partb.c

- 作用: 与 `parta.c` 配合, 展示不同存储类别与链接属性的交互作用。
- 功能: 通过静态变量与外部变量的协作实现部分累计与总和记录; 体现函数内部静态变量 `subtotal` 和文件内部静态变量 `total` 的独立性。
- 演示效果:

```

1  // partb.c -- rest of the program
2  // compile with parta.c
3  #include <stdio.h>
4
5  extern int count;          // reference declaration, external linkage

```



```

6
7  static int total = 0;    // static definition, internal linkage
8  void accumulate(int k); // prototype
9
10 void accumulate(int k) // k has block scope, no linkage
11 {
12     static int subtotal = 0; // static, no linkage
13
14     if (k ≤ 0)
15     {
16         printf("loop cycle: %d\n", count);
17         printf("subtotal: %d; total: %d\n", subtotal, total);
18         subtotal = 0;
19     }
20     else
21     {
22         subtotal += k;
23         total += k;
24     }
25 }

```

```

1  $ gcc parta.c partb.c -o partab && echo "3`n5`n0" | ./partab
2  Enter a positive integer (0 to quit): loop cycle: 1
3  subtotal: 6; total: 6
4  Enter a positive integer (0 to quit): loop cycle: 2
5  subtotal: 15; total: 21
6  Enter a positive integer (0 to quit): Loop executed 2 times

```

diceroll.h

- 作用: 定义与 `diceroll.c` 交互的外部接口。
- 功能: 声明了外部变量 `roll_count` 及函数 `roll_n_dice(int dice, int sides)`，供主程序调用。
- 演示效果:

```

1  //diceroll.h
2  extern int roll_count;
3
4  int roll_n_dice(int dice, int sides);

```

dyn_arr.c

- 作用: 演示如何动态分配数组内存并存储用户输入的 double 类型数据。
- 功能: 使用 `malloc()` 分配指定大小的 double 数组, 读取用户输入并输出所有数据, 最后释放内存。
- 演示效果:

```
1  /* dyn_arr.c -- dynamically allocated array */
2  #include <stdio.h>
3  #include <stdlib.h> /* for malloc(), free() */
4
5  int main(void)
6  {
7      double * ptd;
8      int max;
9      int number;
10     int i = 0;
11
12     puts("What is the maximum number of type double entries?");
13     if (scanf("%d", &max) != 1)
14     {
15         puts("Number not correctly entered -- bye.");
16         exit(EXIT_FAILURE);
17     }
18     ptd = (double *) malloc(max * sizeof (double));
19     if (ptd == NULL)
20     {
21         puts("Memory allocation failed. Goodbye.");
22         exit(EXIT_FAILURE);
23     }
24     /* ptd now points to an array of max elements */
25     puts("Enter the values (q to quit):");
26     while (i < max && scanf("%lf", &ptd[i]) == 1)
27         ++i;
28     printf("Here are your %d entries:\n", number = i);
29     for (i = 0; i < number; i++)
30     {
31         printf("%7.2f ", ptd[i]);
```

```

32         if (i % 7 == 6)
33             putchar('\n');
34     }
35     if (i % 7 != 0)
36         putchar('\n');
37     puts("Done.");
38     free(ptd);
39
40     return 0;
41 }

```

```

1  $ gcc dyn_arr.c -o dyn_arr && echo "5`n1.1`n2.2`n3.3`n4.4`n5.5`nq" | ./dyn_arr
2  What is the maximum number of type double entries?
3  Enter the values (q to quit):
4  Here are your 5 entries:
5      1.10      2.20      3.30      4.40      5.50
6  Done.

```

forc99.c

- 作用: 演示 C99 新的代码块作用域规则, 尤其是 for 循环中变量声明的作用域。
- 功能: 展示同名变量在不同作用域中的地址和值变化, 帮助理解变量遮蔽与作用域生命周期。
- 演示效果:

```

1  // forc99.c -- new C99 block rules
2  #include <stdio.h>
3  int main()
4  {
5      int n = 8;
6
7      printf("    Initially, n = %d at %p\n", n, &n);
8      for (int n = 1; n < 3; n++)
9          printf("        loop 1: n = %d at %p\n", n, &n);
10     printf("After loop 1, n = %d at %p\n", n, &n);
11     for (int n = 1; n < 3; n++)
12     {
13         printf(" loop 2 index n = %d at %p\n", n, &n);
14         int n = 6;
15         printf("        loop 2: n = %d at %p\n", n, &n);
16         n++;

```

```

17     }
18     printf("After loop 2, n = %d at %p\n", n, &n);
19
20     return 0;
21 }

```

```

1  $ gcc forc99.c -o forc99 && ./forc99
2  Initially, n = 8 at 000000be407ffdac
3      loop 1: n = 1 at 000000be407ffda8
4      loop 1: n = 2 at 000000be407ffda8
5  After loop 1, n = 8 at 000000be407ffdac
6      loop 2 index n = 1 at 000000be407ffda4
7      loop 2: n = 6 at 000000be407ffda0
8      loop 2 index n = 2 at 000000be407ffda4
9      loop 2: n = 6 at 000000be407ffda0
10 After loop 2, n = 8 at 000000be407ffdac

```

global.c

- 作用: 演示外部变量（`extern`）的声明与作用范围。
- 功能: 定义了全局变量 `units`，并通过两个函数共享访问，实现输入验证的交互。
- 演示效果:

```

1  /* global.c -- uses an external variable */
2  #include <stdio.h>
3  int units = 0;          /* an external variable      */
4  void critic(void);
5  int main(void)
6  {
7      extern int units; /* an optional redeclaration */
8
9      printf("How many pounds to a firkin of butter?\n");
10     scanf("%d", &units);
11     while ( units != 56)
12         critic();
13     printf("You must have looked it up!\n");
14
15     return 0;
16 }
17

```

```

18 void critic(void)
19 {
20     /* optional redeclaration omitted */
21     printf("No luck, my friend. Try again.\n");
22     scanf("%d", &units);
23 }

```

```

1 $ gcc global.c -o global && echo "12`n56" | ./global
2 How many pounds to a firkin of butter?
3 No luck, my friend. Try again.
4 You must have looked it up!

```

hiding.c

- 作用: 演示变量在不同代码块中的作用域与遮蔽现象。
- 功能: 展示同名变量在外部块、内部块及循环体中的地址和值变化，帮助理解作用域规则。
- 演示效果:

```

1 // hiding.c -- variables in blocks
2 #include <stdio.h>
3 int main()
4 {
5     int x = 30;          // original x
6
7     printf("x in outer block: %d at %p\n", x, &x);
8     {
9         int x = 77;     // new x, hides first x
10        printf("x in inner block: %d at %p\n", x, &x);
11    }
12    printf("x in outer block: %d at %p\n", x, &x);
13    while (x++ < 33) // original x
14    {
15        int x = 100;    // new x, hides first x
16        x++;
17        printf("x in while loop: %d at %p\n", x, &x);
18    }
19    printf("x in outer block: %d at %p\n", x, &x);
20
21    return 0;

```

```

1  $ gcc hiding.c -o hiding && ./hiding
2  x in outer block: 30 at 00000039ce1ffa1c
3  x in inner block: 77 at 00000039ce1ffa18
4  x in outer block: 30 at 00000039ce1ffa1c
5  x in while loop: 101 at 00000039ce1ffa14
6  x in whileloop: 101 at 00000039ce1ffa14
7  x in while loop: 101 at 00000039ce1ffa14
8  x in outerblock: 34 at 00000039ce1ffa1c

```

loc_stat.c

- 作用: 展示局部静态变量与普通局部变量在函数调用间的差别。
- 功能: 重复调用函数 `trystat()`，比较 `fade`（普通局部变量）和 `stay`（静态局部变量）随调用次数的变化情况。
- 演示效果:

```

1  /* loc_stat.c -- using a local static variable */
2  #include <stdio.h>
3  void trystat(void);
4
5  int main(void)
6  {
7      int count;
8
9      for (count = 1; count ≤ 3; count++)
10     {
11         printf("Here comes iteration %d:\n", count);
12         trystat();
13     }
14
15     return 0;
16 }
17
18 void trystat(void)
19 {
20     int fade = 1;
21     static int stay = 1;
22

```

```
23     printf("fade = %d and stay = %d\n", fade++, stay++);
24 }
```

```
1  $ gcc loc_stat.c -o loc_stat && ./loc_stat
2  Here comes iteration 1:
3  fade = 1 and stay = 1
4  Here comes iteration 2:
5  fade = 1 and stay = 2
6  Here comes iteration 3:
7  fade = 1 and stay = 3
```

manydice.c

- 作用: 演示如何使用 `diceroll.c` 提供的函数进行多次掷骰子模拟。
- 功能: 接收用户输入的骰子面数和数量, 调用 `roll_n_dice()` 计算总点数, 并统计函数调用次数。
- 演示效果:

```
1  /* manydice.c -- multiple dice rolls */
2  /* compile with diceroll.c */
3  #include <stdio.h>
4  #include <stdlib.h> /* for library srand() */
5  #include <time.h> /* for time() */
6  #include "diceroll.h" /* for roll_n_dice() */
7  /* and for roll_count */
8  int main(void)
9  {
10     int dice, roll;
11     int sides;
12     int status;
13
14     srand((unsigned int) time(0)); /* randomize seed */
15     printf("Enter the number of sides per die, 0 to stop.\n");
16     while (scanf("%d", &sides) == 1 && sides > 0 )
17     {
18         printf("How many dice?\n");
19         if ((status = scanf("%d", &dice)) != 1)
20         {
21             if (status == EOF)
22                 break; /* exit loop */

```

```

23         else
24         {
25             printf("You should have entered an integer.");
26             printf(" Let's begin again.\n");
27             while (getchar() != '\n')
28                 continue; /* dispose of bad input */
29             printf("How many sides? Enter 0 to stop.\n");
30             continue; /* new loop cycle */
31         }
32     }
33     roll = roll_n_dice(dice, sides);
34     printf("You have rolled a %d using %d %d-sided dice.\n",
35           roll, dice, sides);
36     printf("How many sides? Enter 0 to stop.\n");
37 }
38 printf("The rollem() function was called %d times.\n",
39       roll_count); /* use extern variable */
40
41 printf("GOOD FORTUNE TO YOU!\n");
42
43 return 0;
44 }

```

```

1  $ gcc manydice.c diceroll.c -o manydice && echo "6`n3`n0" | ./manydice
2  Enter the number of sides per die, 0 to stop.
3  How many dice?
4  You have rolled a 7 using 3 6-sided dice.
5  How many sides? Enter 0 to stop.
6  The rollem() function was called 3 times.
7  GOOD FORTUNE TO YOU!
8

```